THE SKY GEN
AI ENGINEER

PROMPT ENGINEERING TOOLKIT

Tested Prompt Library for Python, Machine Learning & AI Developme

PROJECT 01 — BEGINNER LEVEL

PROFESSION	AI Engineer
SPECIALIZATION	Python & Machine Learning
PROGRAM	September–November 2026
PREPARED FOR	The Sky Gen

Executive Summary

This project develops a practical prompt library for an AI Engineer working primarily with Python and Machine Learning. It covers code generation, debugging, preprocessing, feature engineering, model selection, evaluation, error analysis, documentation, pipeline design and deployment planning. Each prompt is tested before and after improvement using the same customer-churn project context.

CORE TASK COVERAGE

Python code generation • debugging • code explanation • data preprocessing • feature engineering • model selection • model evaluation • hyperparameter tuning • ML error analysis • data leakage detection • SQL/data analysis • technical documentation • ML pipeline architecture • deployment planning • technical project review.

ENGINEERING PRINCIPLE

Use AI to accelerate engineering work, not to replace engineering verification.

Prompt Framework, Testing Methodology & Few-Shot Evidence

The same structure is applied across the library so prompt improvements can be inspected consistently.

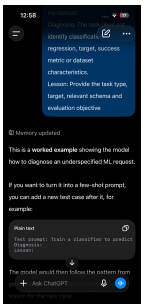
ROLE	Defines the expertise and perspective expected from the AI.
TASK	States the exact engineering action to perform.
CONTEXT	Provides the project, dataset and objective.
INPUT	Supplies schema, code, errors, workflow or other evidence.
OUTPUT FORMAT	Specifies the required structure, table, checklist or code format.
CONSTRAINTS	Controls leakage, hallucination, assumptions, scope and reproducibility.

TESTING PROCEDURE

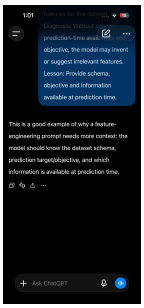
Run baseline → capture output → run improved prompt with comparable information → capture output → compare specificity, assumptions, structure, verification and usefulness.

FEW-SHOT EXAMPLES

Worked examples demonstrate ML error classification, code-review finding format and data-leakage review format.



FEW-SHOT 1



FEW-SHOT 2



FEW-SHOT 3

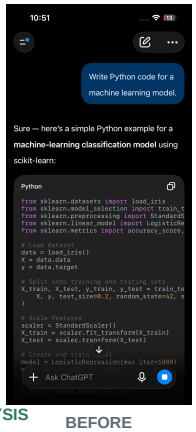
Prompt 01 — Python Code Generation

BASELINE

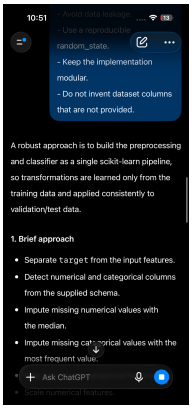
Write a python code for machine learning model.

STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a senior Python and machine-learning engineer. TASK: Create a Python training pipeline for a binary classification problem. CONTEXT: The dataset is a customer churn dataset containing numerical and categorical features. The target column is "churn". Missing values may exist in both numerical and categorical columns. INPUT: Dataset schema: - customer_age: numerical - monthly_charges: numerical - contract_type: categorical - tenure_months: numerical - support_tickets: numerical - payment_method: categorical - churn: binary target - Rows: approximately 5,000 OUTPUT FORMAT: 1. Brief approach 2. Complete Python code 3. Explanation of preprocessing 4. Evaluation strategy 5. Assumptions CONSTRAINTS: - Use scikit-learn. - Use Pipeline and ColumnTransformer where appropriate. - Avoid data leakage. - Use a reproducible random_state. - Keep the implementation modular. - Do not invent dataset columns that are not provided.



BEFORE



AFTER

ANALYSIS

The improved prompt supplies task context, dataset assumptions, implementation requirements, output structure and engineering constraints. This makes the requested result more specific, easier to inspect and less dependent on unstated assumptions.

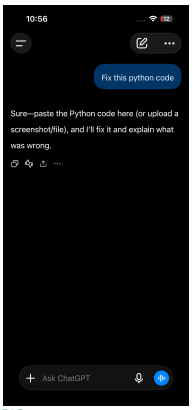
Prompt 02 — Python Debugging

BASELINE

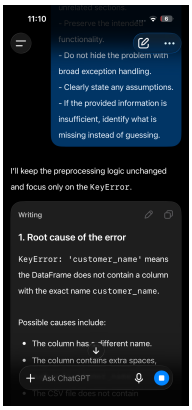
Fix this python code.

STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a senior Python debugging engineer. TASK: Diagnose and correct the following Python code. CONTEXT: The code is part of a machine-learning preprocessing workflow. CODE: [import pandas as pd df = pd.read_csv("customers.csv") df["age"] = df["age"].fillna(df["age"].median()) print(df["customer_name"])] ERROR: [KeyError: 'customer_name'] OUTPUT FORMAT: 1. Root cause of the error 2. Exact problematic section 3. Corrected code 4. Explanation of why the correction works 5. Prevention strategy CONSTRAINTS: - Do not rewrite unrelated sections. - Preserve the intended functionality. - Do not hide the problem with broad exception handling. - Clearly state any assumptions. - If the provided information is insufficient, identify what is missing instead of guessing.



BEFORE



AFTER

ANALYSIS

The improved version separates diagnosis from correction and requires the model to explain the root cause. It also prevents unrelated rewrites and encourages explicit handling of missing information.

Prompt 03 — Explain ML Code

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

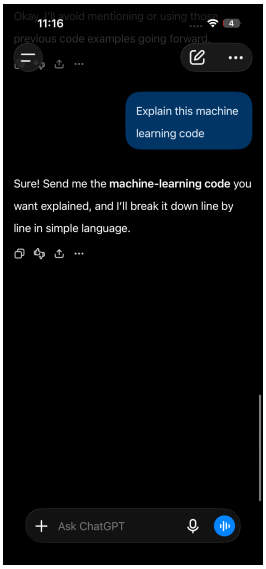
BASELINE / WEAK PROMPT

Explain this machine learning code.

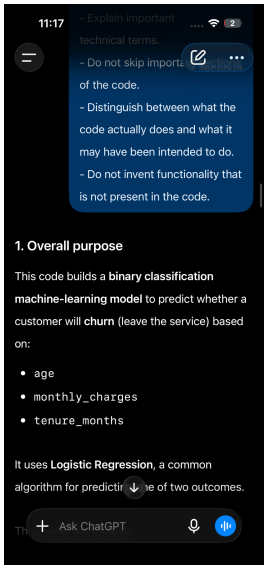
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as an experienced machine-learning mentor. TASK: Explain the supplied Python machine-learning code to a Python developer who is developing their ML knowledge. CODE: import pandas as pd from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.linear_model import LogisticRegression from sklearn.metrics import accuracy_score df = pd.read_csv("customers.csv") X = df[["customer_age", "monthly_charges", "tenure_months"]] y = df["churn"] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) scaler = StandardScaler() X_train = scaler.fit_transform(X_train) X_test = scaler.transform(X_test) model = LogisticRegression() model.fit(X_train, y_train) predictions = model.predict(X_test) accuracy = accuracy_score(y_test, predictions) print("Model Accuracy:", accuracy) OUTPUT FORMAT: 1. Overall purpose 2. Step-by-step explanation 3. Important machine-learning concepts used 4. Input and output flow 5. Potential problems or risks 6. Short summary CONSTRAINTS: - Explain important technical terms. - Do not skip important sections of the code. - Distinguish between what the code actually does and what it may have been intended to do. - Do not invent functionality that is not present in the code.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt defines the audience, requested explanation structure and an important distinction between actual behavior and intended behavior. This reduces vague or overly generic explanations.

Prompt 04 — Data Preprocessing

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

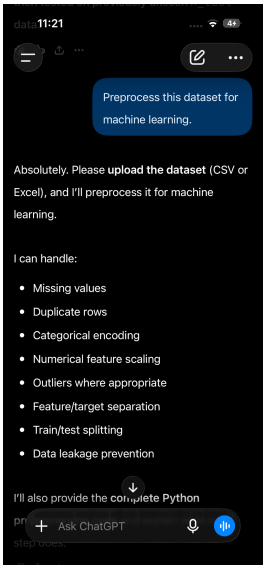
BASELINE / WEAK PROMPT

Preprocess this dataset for machine learning.

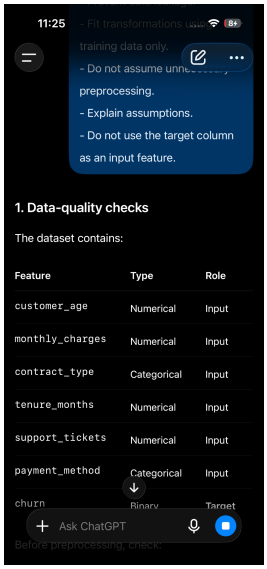
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a senior machine-learning engineer specializing in data preprocessing. TASK: Design an appropriate preprocessing strategy for the dataset below. CONTEXT: The goal is to prepare a customer churn dataset for a binary classification model. DATASET: Dataset: Customer Churn Dataset Rows: approximately 5,000 Columns: customer_age Type: numerical Example: 35 monthly_charges Type: numerical Example: 79.50 contract_type Type: categorical Possible values: Month-to-month, One-year, Two-year tenure_months Type: numerical Example: 8 support_tickets Type: numerical Example: 4 payment_method Type: categorical Possible values: Electronic, Bank transfer, Credit card, Mailed check churn Type: binary target 0 = Did not churn 1 = Churned Missing values may exist in both numerical and categorical columns. OUTPUT FORMAT: 1. Data-quality checks 2. Missing-value strategy 3. Categorical-feature treatment 4. Numerical-feature treatment 5. Train/test preprocessing strategy 6. Recommended scikit-learn implementation 7. Reasoning behind each decision CONSTRAINTS: - Prevent data leakage. - Fit transformations using training data only. - Do not assume unnecessary preprocessing. - Explain assumptions. - Do not use the target column as an input feature.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved version identifies the target, requests specific preprocessing categories and makes leakage prevention an explicit constraint. It also asks the model to justify rather than blindly apply transformations.

Prompt 05 — Feature Engineering

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

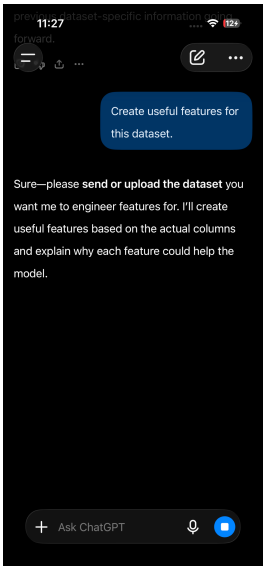
BASELINE / WEAK PROMPT

Suggest some features for my dataset.

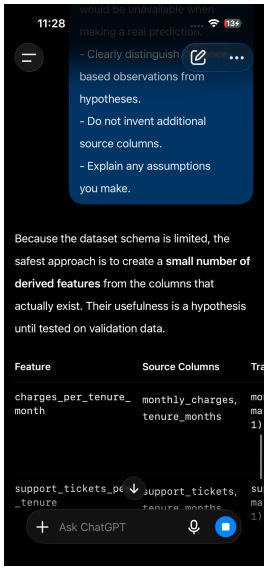
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a machine-learning feature-engineering specialist. TASK: Identify potentially useful features that could improve a customer-churn classification model. CONTEXT: The dataset is a customer churn dataset. The goal is to predict whether a customer will churn using information available at prediction time. DATASET: Dataset: Customer Churn Dataset Rows: approximately 5,000 Available columns: customer_age monthly_charges contract_type tenure_months support_tickets payment_method churn Target: churn Target values: 0 = Did not churn 1 = Churned Prediction goal: Predict whether a customer will churn using information available at prediction time. OUTPUT FORMAT: | Feature | Source Columns | Transformation | Expected Usefulness | Risk | After the table, provide: 1. Top candidate features 2. Reasoning behind each candidate 3. Validation approach CONSTRAINTS: - Do not use the target column to create features. - Avoid target leakage. - Do not use information that would be unavailable when making a real prediction. - Clearly distinguish evidence-based observations from hypotheses. - Do not invent additional source columns. - Explain any assumptions you make.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The table format makes the recommendations easier to review. The constraints also focus attention on leakage and on the difference between evidence and hypotheses.

Prompt 06 — Model Selection

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

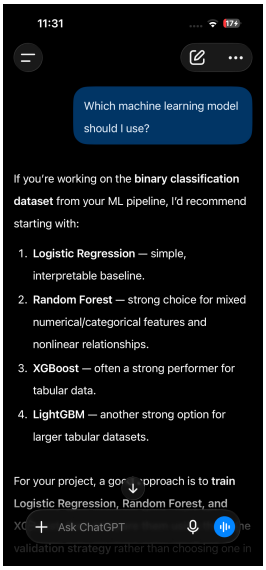
BASELINE / WEAK PROMPT

Which machine learning model should I use?

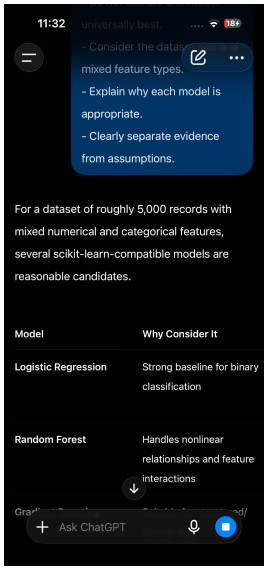
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a machine-learning model-selection specialist. TASK: Recommend suitable candidate models for a binary customer-churn classification problem. CONTEXT: We want to predict whether a customer will churn. The dataset contains approximately 5,000 customer records. The features include both numerical and categorical variables. Missing values may exist. DATASET: - customer_age: numerical - monthly_charges: numerical - contract_type: categorical - tenure_months: numerical - support_tickets: numerical - payment_method: categorical - churn: binary target TARGET: churn 0 = Did not churn 1 = Churned REQUIREMENTS: The models should be suitable for a Python and scikit-learn workflow. OUTPUT FORMAT: Create a comparison table with: | Model | Why Consider It | Strengths | Weaknesses | Computational Considerations | Then provide: 1. Recommended candidate models 2. Suggested evaluation strategy 3. Factors that should influence the final selection 4. Assumptions CONSTRAINTS: - Include at least three reasonable candidate models. - Do not declare one model universally best. - Consider the dataset size and mixed feature types. - Explain why each model is appropriate. - Clearly separate evidence from assumptions.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt replaces an unsupported request for a universal “best” model with a comparative engineering analysis tied to dataset characteristics and experiments.

Prompt 07 — Model Evaluation

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

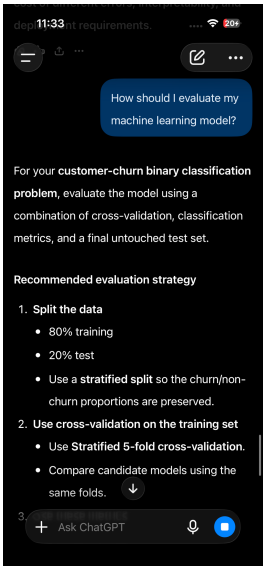
BASELINE / WEAK PROMPT

How should I evaluate my machine learning model?

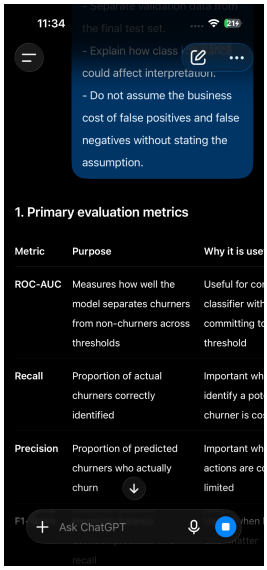
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a machine-learning model-evaluation specialist. TASK: Design an evaluation strategy for a binary customer-churn classification model. CONTEXT: The model predicts whether a customer will churn. The dataset contains approximately 5,000 observations. The target is binary, and the classes may not be perfectly balanced. DATASET: - customer_age: numerical - monthly_charges: numerical - contract_type: categorical - tenure_months: numerical - support_tickets: numerical - payment_method: categorical - churn: binary target TARGET: churn 0 = Did not churn 1 = Churned MODEL: A Logistic Regression classifier trained using scikit-learn. OUTPUT FORMAT: 1. Primary evaluation metrics 2. Secondary metrics 3. Validation strategy 4. What each metric reveals 5. How to interpret the results 6. Common evaluation mistakes 7. Example Python evaluation code CONSTRAINTS: - Do not rely on accuracy alone. - Explain why each recommended metric is useful. - Separate validation data from the final test set. - Explain how class imbalance could affect interpretation. - Do not assume the business cost of false positives and false negatives without stating the assumption.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt provides the information needed to select meaningful metrics and explicitly prevents an automatic accuracy-first recommendation.

Prompt 08 — Hyperparameter Tuning

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

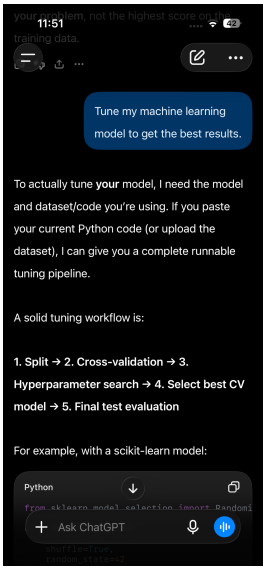
BASELINE / WEAK PROMPT

Tune my machine learning model to get the best results.

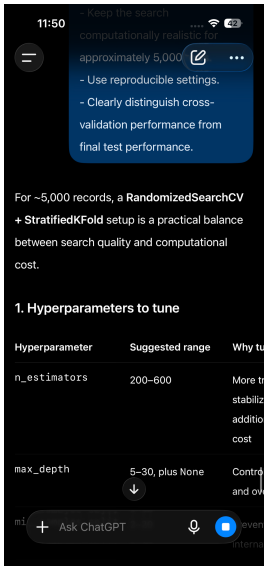
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a senior machine-learning experimentation engineer. TASK: Design a reproducible hyperparameter-tuning strategy for a Random Forest binary classification mod CONTEXT: The dataset contains approximately 5,000 customer records with numerical and categorical features. DATASET: - customer_age: numerical - monthly_charges: numerical - contract_type: categorical - tenure_months: numerical - support_tickets: numerical - payment_method: categorical - churn: binary target TARGET: churn 0 = Did not churn 1 = Churned MODEL: RandomForestClassifier from scikit-learn. OUTPUT FORMAT: 1. Hyperparameters to tune 2. Reasonable search ranges 3. Search method 4. Cross-validation strategy 5. Scoring metric 6. Reproducibility settings 7. Complete Python example 8. Final-test procedure CONSTRAINTS: - Do not use the final test set for hyperparameter tuning. - Explain why each parameter is being tuned. - Keep the search computationally realistic for approximately 5,000 rows. - Use reproducible settings. - Clearly distinguish cross-validation performance from final test performance.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt makes experimental boundaries explicit. It separates tuning from final testing and asks for reproducibility and computational considerations.

Prompt 09 — ML Error Analysis

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

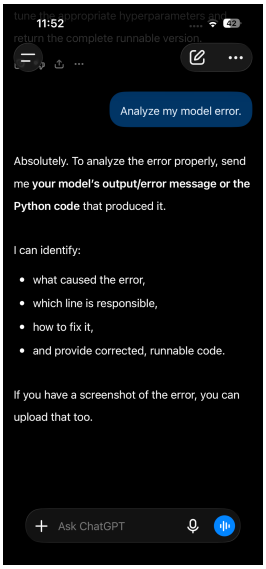
BASELINE / WEAK PROMPT

Analyze my model error.

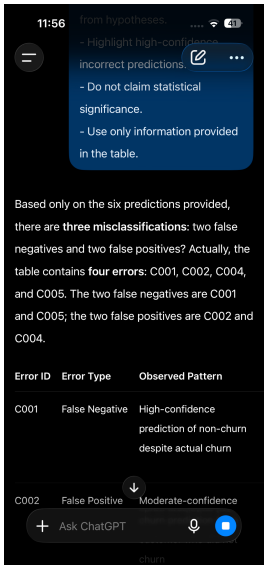
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a machine-learning error-analysis specialist. TASK: Analyze the prediction errors made by a customer-churn classification model and identify patterns that deserve further investigation. CONTEXT: The model predicts whether a customer will churn. DATASET: - customer_age: numerical - monthly_charges: numerical - contract_type: categorical - tenure_months: numerical - support_tickets: numerical - payment_method: categorical - churn: binary target TARGET: churn 0 = Did not churn 1 = Churned MODEL PREDICTIONS: | Customer | Actual | Predicted | Confidence | Contract Type | Monthly Charges | Tenure Months | Support Tickets | |---|---|---|---|---|---|---|---|---| | C001 | 1 | 0 | 0.91 | Month-to-month | 85 | 4 | 6 | | C002 | 0 | 1 | 0.62 | One-year | 55 | 24 | 1 | | C003 | 1 | 1 | 0.88 | Month-to-month | 92 | 3 | 7 | | C004 | 0 | 1 | 0.79 | Two-year | 48 | 36 | 0 | | C005 | 1 | 0 | 0.87 | Month-to-month | 78 | 6 | 5 | | C006 | 0 | 0 | 0.94 | Two-year | 45 | 40 | 1 | OUTPUT FORMAT: Create a table: | Error ID | Error Type | Observed Pattern | Evidence | Suggested Investigation | Then provide: 1. Most important observations 2. High-confidence incorrect predictions 3. Possible hypotheses 4. Recommended next investigations CONSTRAINTS: - Do not claim causation from this small sample. - Clearly separate observations from hypotheses. - Highlight high-confidence incorrect predictions. - Do not claim statistical significance. - Use only information provided in the table.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt creates a consistent error-analysis record and explicitly separates evidence from hypotheses, reducing unsupported causal claims.

Prompt 10 — Data Leakage Detection

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

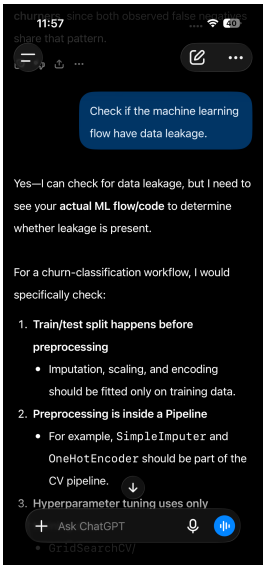
BASELINE / WEAK PROMPT

Check if the machine learning flow have data leakage.

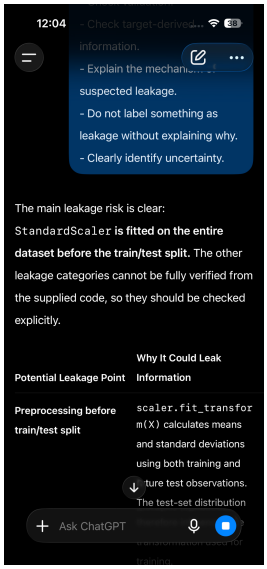
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a machine-learning validation reviewer. TASK: Audit the following machine-learning workflow for potential data leakage. CONTEXT: The workflow predicts customer churn using Python and scikit-learn. DATASET: The dataset contains these numerical features: - customer_age - monthly_charges - tenure_months - support_tickets Target: - churn TARGET: churn 0 = Did not churn 1 = Churned WORKFLOW CODE: import pandas as pd from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.linear_model import LogisticRegression df = pd.read_csv("customers.csv") X = df[["customer_age", "monthly_charges", "tenure_months", "support_tickets"]] y = df["churn"] # Preprocessing is performed before the train/test split scaler = StandardScaler() X_scaled = scaler.fit_transform(X) # Train/test split X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42) # Train model model = LogisticRegression() model.fit(X_train, y_train) predictions = model.predict(X_test) OUTPUT FORMAT: Create a table: | Potential Leakage Point | Why It Could Leak Information | Severity | Safer Alternative | Verification Step | Then provide: 1. Overall assessment 2. Explanation of each potential issue 3. Corrected workflow 4. Verification checklist CONSTRAINTS: - Check the train/test split. - Check preprocessing. - Check feature engineering. - Check validation. - Check target-derived information. - Explain the mechanism of suspected leakage. - Do not label something as leakage without explaining why. - Clearly identify uncertainty.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt gives the reviewer a systematic audit structure and specifies where leakage should be checked. It also requires the mechanism and verification step to be explained.

Prompt 11 — SQL / Data Analysis

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

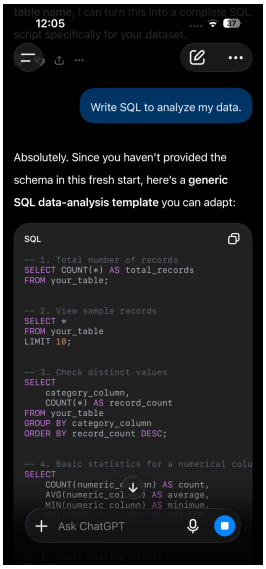
BASELINE / WEAK PROMPT

Write SQL to analyze my data.

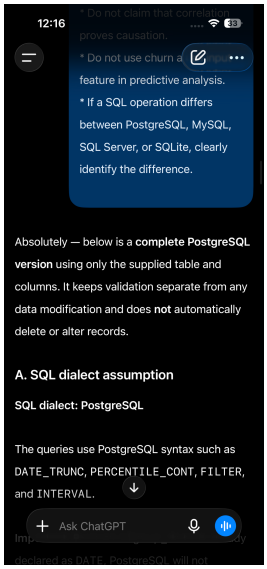
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a senior SQL data analyst. TASK: Write complete, runnable SQL to clean, validate, and analyze a customer-churn dataset. DATABASE TABLE: customer_churn SCHEMA: ● customer_id INTEGER ● customer_age INTEGER ● contract_type VARCHAR ● monthly_charges DECIMAL(10,2) ● tenure_months INTEGER ● support_tickets INTEGER ● payment_method VARCHAR ● churn INTEGER ● signup_date DATE TARGET: churn is the binary target: ● 0 = retained ● 1 = churned REQUIREMENTS: 1. DATA QUALITY CHECKS Write SQL to: ● Count total rows. ● Check for NULL values in every column. ● Detect duplicate customer_id values. ● Check for invalid churn values other than 0 or 1. ● Check for invalid or negative customer_age values. ● Check for invalid or negative monthly_charges. ● Check for invalid or negative tenure_months. ● Check for invalid or negative support_tickets. ● Check for invalid signup_date values where supported by the SQL dialect. 2. DATA CLEANING Provide SQL approaches for: ● Handling NULL values. ● Removing or identifying duplicate customer records. ● Handling invalid numeric values. ● Standardizing categorical values such as contract_type and payment_method. Do not silently delete records. Clearly show which records would be affected before providing any DELETE or UPDATE statement. 3. CUSTOMER-CHURN ANALYSIS Write SQL to calculate: ● Total customers. ● Churned customers. ● Retained customers. ● Overall churn rate. ● Churn rate by contract_type. ● Churn rate by payment_method. ● Churn rate by tenure group. ● Churn rate by support-ticket group. ● Average monthly_charges for churned versus retained customers. ● Average tenure_months for churned versus retained customers. ● Monthly customer signups using signup_date. 4. OUTLIER / ANOMALY CHECKS Write SQL queries to identify potentially unusual values in: ● monthly_charges ● tenure_months ● support_tickets ● customer_age Do not automatically remove outliers. Flag them for review. 5. OUTPUT FORMAT Provide: A. SQL dialect assumption. B. Data-quality queries. C. Data-cleaning queries. D. Exploratory-analysis queries. E. Anomaly-detection queries. F. Brief explanation of each query. G. Important assumptions and limitations. CONSTRAINTS: ● Use only the supplied table and column names. ● Make the SQL runnable for the stated SQL dialect. ● Do not invent additional columns. ● Do not modify data unless explicitly requested. ● Clearly separate SELECT-based validation from UPDATE/DELETE operations. ● Handle NULL values appropriately. ● Do not claim that correlation proves causation. ● Do not use churn as an input feature in predictive analysis. ● If a SQL operation differs between PostgreSQL, MySQL, SQL Server, or SQLite, clearly identify the difference.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved version supplies schema, question, output requirements and validation concerns. It directly addresses a common failure mode: inventing columns or making hidden database assumptions.

Prompt 12 — Technical Documentation

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

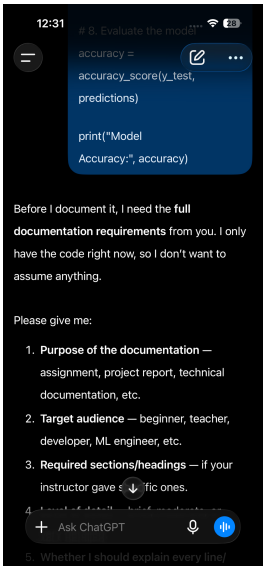
BASELINE / WEAK PROMPT

Document this code.

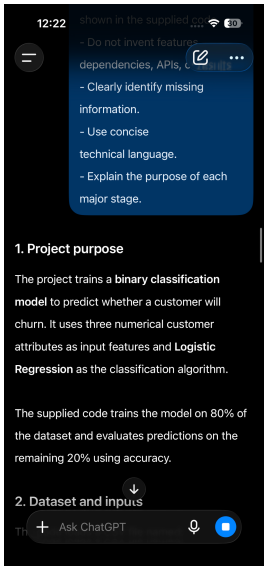
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a technical writer with professional Python and machine-learning engineering experience. TASK: Create technical documentation for the following customer-churn machine-learning project. CONTEXT: The project uses Python and scikit-learn to train a binary classification model that predicts whether a customer will churn. PROJECT CODE: import pandas as pd from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.linear_model import LogisticRegression from sklearn.metrics import accuracy_score # 1. Load the dataset df = pd.read_csv("customers.csv") # 2. Check that required columns exist required_columns = ["customer_age", "monthly_charges", "tenure_months", "churn"] missing_columns = [col for col in required_columns if col not in df.columns] if missing_columns: raise ValueError(f"Missing required columns: {missing_columns}") # 3. Select features and target X = df[["customer_age", "monthly_charges", "tenure_months"]] y = df["churn"] # 4. Split the data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # 5. Standardize the numerical features scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # 6. Create and train the Logistic Regression model model = LogisticRegression(random_state=42) model.fit(X_train_scaled, y_train) # 7. Make predictions predictions = model.predict(X_test_scaled) # 8. Evaluate the model accuracy = accuracy_score(y_test, predictions) print("Model Accuracy:", accuracy) AUDIENCE: Python developers and ML engineers who may need to understand or maintain this project. OUTPUT FORMAT: 1. Project purpose 2. Dataset and inputs 3. Target variable 4. Outputs 5. Workflow 6. Dependencies 7. Assumptions 8. Evaluation method 9. Error considerations 10. Usage instructions 11. Maintenance notes CONSTRAINTS: - Document only behavior shown in the supplied code. - Do not invent features, dependencies, APIs, or results. - Clearly identify missing information. - Use concise technical language. - Explain the purpose of each major stage.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt defines the audience and documentation sections while explicitly preventing invented behavior. This makes the output more useful as engineering documentation.

Prompt 13 — ML Pipeline Architecture

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

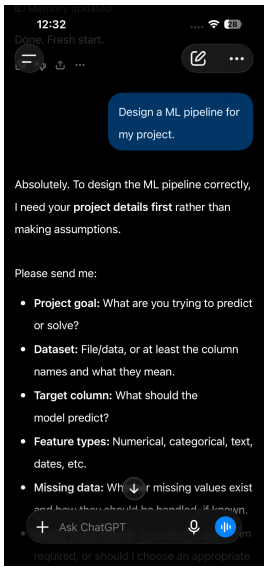
BASELINE / WEAK PROMPT

Design a ML pipeline for my project.

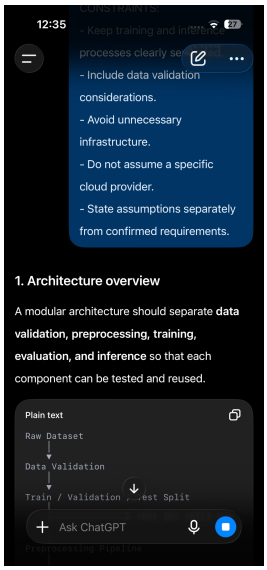
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a senior machine-learning systems engineer. TASK: Design a modular machine-learning pipeline architecture for a binary classification project. PROJECT CONTEXT: - Dataset: approximately 5,000 rows - Numerical and categorical features - Binary target - Python and scikit-learn - Need reproducible training and evaluation - Future deployment may be required OUTPUT FORMAT: 1. Architecture overview 2. Pipeline stages 3. Data flow 4. Train/validation/test separation 5. Model-training process 6. Monitoring considerations 7. Potential failure points 8. Suggested project structure CONSTRAINTS: - Keep training and inference processes clearly separated. - Include data validation considerations. - Avoid unnecessary infrastructure. - Do not assume a specific cloud provider. - State assumptions separately from confirmed requirements.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt provides architecture-relevant context and forces separation of training, inference, validation and monitoring. It also keeps the proposed system proportional to the stated requirements.

Prompt 14 — Deployment Planning

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

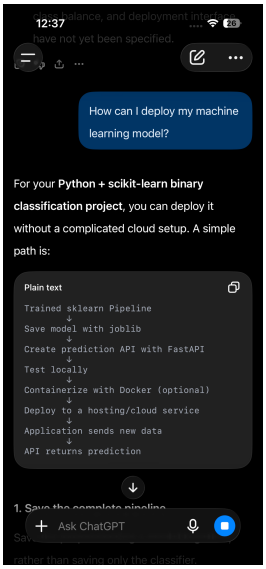
BASELINE / WEAK PROMPT

How can I deploy my machine learning model?

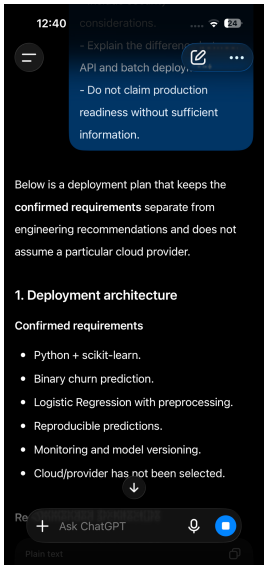
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as an ML deployment engineer. TASK: Create a deployment plan for a customer-churn machine-learning model. CONTEXT: The model is a binary classification model built with Python and scikit-learn. It predicts whether a customer is likely to churn. DATASET: - customer_age: numerical - monthly_charges: numerical - contract_type: categorical - tenure_months: numerical - support_tickets: numerical - payment_method: categorical - churn: binary target MODEL: Logistic Regression with preprocessing. INPUT EXAMPLE: customer_age = 35 monthly_charges = 79.50 contract_type = "Month-to-month" tenure_months = 8 support_tickets = 4 payment_method = "Electronic" OUTPUT: A binary churn prediction. REQUIREMENTS: - Predictions should be reproducible. - The model and preprocessing steps must remain consistent. - The deployment environment has not yet been selected. - The system should support monitoring and model versioning. OUTPUT FORMAT: 1. Deployment architecture 2. Model packaging 3. API option 4. Batch-processing option 5. Input validation 6. Monitoring 7. Model versioning 8. Data/version tracking 9. Rollback strategy 10. Testing checklist CONSTRAINTS: - Do not assume a specific cloud provider. - Separate confirmed requirements from recommendations. - Include security considerations. - Explain the difference between API and batch deployment. - Do not claim production readiness without sufficient information.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt turns a broad deployment question into a concrete planning exercise and requires operational considerations such as monitoring, versioning, rollback and testing.

Prompt 15 — Technical Project Review

Before/after evidence retained from the submitted document; screenshots are displayed smaller for the 16-page format.

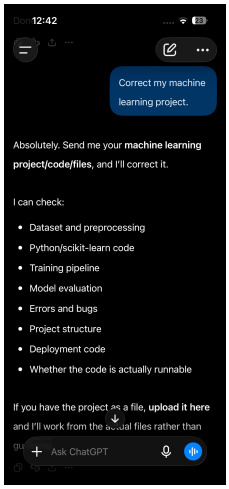
BASELINE / WEAK PROMPT

Review my machine learning project and tell me if its correct.

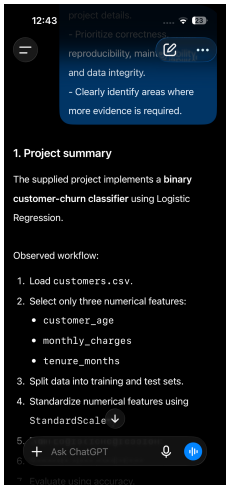
IMPROVED STRUCTURED PROMPT

Improved Structured Prompt ROLE: Act as a senior AI engineer conducting a technical project review. TASK: Review the supplied customer-churn machine-learning project and identify technical strengths, risks, and priority improvements. PROJECT OBJECTIVE: Build a binary classification model that predicts whether a customer will churn. DATASET: - customer_age: numerical - monthly_charges: numerical - contract_type: categorical - tenure_months: numerical - support_tickets: numerical - payment_method: categorical - churn: binary target PROJECT CODE: import pandas as pd from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.linear_model import LogisticRegression from sklearn.metrics import accuracy_score df = pd.read_csv("customers.csv") X = df[["customer_age", "monthly_charges", "tenure_months"]] y = df["churn"] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) scaler = StandardScaler() X_train = scaler.fit_transform(X_train) X_test = scaler.transform(X_test) model = LogisticRegression() model.fit(X_train, y_train) predictions = model.predict(X_test) accuracy = accuracy_score(y_test, predictions) print("Model Accuracy:", accuracy) OUTPUT FORMAT: 1. Project summary 2. Technical strengths 3. Machine-learning methodology 4. Code-quality observations 5. Reproducibility observations 6. Technical risks 7. Priority improvements 8. Questions that should be answered before production use CONSTRAINTS: - Base observations only on the supplied material. - Distinguish observed issues from recommendations. - Do not invent missing project details. - Prioritize correctness, reproducibility, maintainability, and data integrity. - Clearly identify areas where more evidence is required.

BEFORE / AFTER TEST EVIDENCE



BEFORE



AFTER

COMPARISON & IMPROVEMENT ANALYSIS

The improved prompt creates a repeatable review framework and prevents unsupported assumptions. It also separates observed issues from recommendations, making the review easier to act on.

REQUIRED APPENDICES — CONDENSED

Failure Log: 01 Ambiguous ML Task — missing task type, target, metric and schema. 02 Missing Dataset Context — feature suggestions may be invented or irrelevant. 03 Conflicting Constraints — fastest, most accurate and no computation are not simultaneously measurable. 04 Missing Output Format — “Analyze my model” gives no required structure. 05 Unclear Audience — “Explain this ML result” does not define technical level or use case.

Limitations & Verification: AI may hallucinate APIs, produce incorrect/incomplete code, miss data leakage, use outdated information, make unsupported assumptions or give confidently wrong explanations. Verify by running code, checking current documentation, auditing leakage, confirming assumptions and ensuring the engineer can explain and defend the result.

One-Page Usage Guide: Select task → replace placeholders → add real context → run/capture output → inspect → verify → iterate. Core formula: Role + Task + Context + Input + Output Format + Constraints.

Final checklist: 15 tested prompts • before/after evidence • comparison analysis • 3 few-shot examples • 5 failure cases • limitations • usage guide • AI tool/model/date metadata • no personal/confidential information.